

RAPPORT TECHNIQUE

Projet : MboaLearn — Mini Learning Management System (LMS)

Analyse fonctionnelle, architecturale et technologique du projet

Unité d'Enseignement : INF222 — Programmation Web

Examineur : Dr Messi Nguele

Étudiant(e) : Yvana Emilia Lalane Larcier

Matricule : 24G2439

Établissement : Université de Yaoundé I — Faculté des Sciences

Environnement d'exécution : Hébergement local via XAMPP (Apache + MySQL + PHP)

Date : 2026

1. Présentation générale du projet

MboaLearn est une plateforme web d'apprentissage en ligne (mini LMS) développée dans le cadre du TP de l'UE INF222 - Programmation Web. Elle permet à des étudiants de suivre des cours composés de leçons (vidéo ou PDF), de passer des évaluations sous forme de QCM, de suivre leur progression et d'obtenir des certificats automatiquement lorsque le score d'une évaluation est supérieur ou égal à 50%.

Le projet illustre une application web dynamique complète, combinant une partie front-end (HTML, CSS, JavaScript, AJAX) et une partie back-end (PHP avec PDO et MySQL), exécutée localement à l'aide de XAMPP.

1.1 Objectifs pédagogiques du TP

- Mettre en pratique la création de pages dynamiques avec PHP et l'accès à une base de données via PDO.
- Implémenter un système d'authentification et de gestion de sessions (connexion, inscription, déconnexion).
- Gérer plusieurs rôles utilisateurs avec des permissions différenciées (étudiant, enseignant, promoteur).
- Utiliser AJAX (Fetch API et XMLHttpRequest) pour des interactions sans rechargement de page.
- Concevoir une base de données relationnelle cohérente avec des clés étrangères et des contraintes.
- Structurer un projet web avec des fichiers réutilisables (header, footer, includes).

2. Architecture du projet

Le projet suit une architecture procédurale typique d'une application PHP « classique » (sans framework), organisée par dossiers selon le rôle de l'utilisateur et le type de fichier.

2.1 Arborescence des fichiers

```
L2_LMS/
├─ index.html           (page d'accueil publique)
├─ login.php / register.php / logout.php
├─ dashboard.php        (tableau de bord, redirige selon le rôle)
├─ cours.php / lecon.php (catalogue de cours et lecture des leçons)
├─ evaluation.php        (passage d'un QCM)
├─ resultats.php         (résultats d'une évaluation)
├─ certificats.php / certificate.php
├─ config/
│   └─ database.php      (connexion PDO, configuration BD)
├─ includes/
│   └─ auth.php           (fonctions d'authentification et de rôles)
│   └─ header.php         (en-tête commun / navigation)
│   └─ footer.php         (pied de page commun)
```

```

├─ ajax/
|   ├─ update_progress.php  (MAJ progression d'une leçon)
|   ├─ submit_quiz.php      (soumission d'un QCM)
|   └─ get_lessons.php      (récupération dynamique des leçons)
├─ enseignant/              (espace Enseignant)
├─ promoteur/              (espace Promoteur)
├─ assets/
|   ├─ css/style.css
|   ├─ js/app.js
|   └─ images/mboalearn.png
├─ index.sql                (schéma de la base de données)
└─ migration_certificats.sql

```

2.2 Architecture logique (3 couches)

Couche	Rôle	Fichiers concernés
Présentation (front-end)	Affichage des pages, mise en forme, interactivité côté navigateur	*.html, assets/css/style.css, assets/js/app.js
Logique applicative (back-end)	Traitement des requêtes, authentification, règles métier, génération du HTML dynamique	*.php, includes/, ajax/, enseignant/, promoteur/
Données	Stockage persistant via MySQL, accès via PDO	config/database.php, index.sql

2.3 Modèle de rôles et contrôle d'accès

L'application repose sur trois rôles définis dans la table users (champ role) : étudiant, enseignant et promoteur. Le fichier includes/auth.php centralise la logique de sécurité :

- `is_logged_in()` : vérifie la présence d'un identifiant de session (`$_SESSION['user_id']`).
- `has_role($role)` : compare le rôle stocké en session à un rôle attendu.
- `require_login()` : redirige vers `login.php` si l'utilisateur n'est pas connecté.
- `require_role($role)` : redirige vers `dashboard.php` si l'utilisateur n'a pas le rôle requis.
- `get_user_info()` : récupère les informations complètes de l'utilisateur via une requête PDO préparée.

`dashboard.php` agit comme un routeur central : selon le rôle de l'utilisateur connecté, il redirige automatiquement vers `enseignant/dashboard.php`, `promoteur/dashboard.php`, ou affiche le tableau de bord de l'étudiant.

2.4 Base de données (MySQL)

Le schéma (`index.sql`) définit 9 tables principales reliées par des clés étrangères, formant un modèle relationnel cohérent :

Table	Rôle principal
-------	----------------

users	Comptes utilisateurs (nom, email, mot de passe haché, rôle)
modules	Regroupement thématique de cours
cours	Cours, liés à un module et à un enseignant
lecons	Leçons (vidéo ou PDF) d'un cours, ordonnées
evaluations	QCM associés à une leçon
questions	Questions d'une évaluation avec leur bonne réponse
reponses	Réponses données par chaque étudiant
resultats	Scores obtenus par les étudiants aux évaluations
progression	Suivi de l'avancement d'un étudiant sur chaque leçon
certificats	Certificats générés automatiquement (score $\geq$ 50%)

Les mots de passe sont stockés sous forme hachée via `password_hash()` / vérifiés avec `password_verify()`, et toutes les requêtes utilisent des requêtes préparées PDO (paramètres liés par '?'), ce qui protège contre les injections SQL.

3. Technologies utilisées

Technologie	Rôle dans MboaLearn
HTML5	Structure des pages (index.html, formulaires, sections du contenu généré par PHP)
CSS3	Mise en forme via <code>assets/css/style.css</code> : grilles (grid-2, grid-3), cartes, barres de progression, badges de rôle, mise en page responsive
JavaScript (ES6+)	Interactivité côté client : validation de formulaires, notifications, lecteur vidéo avec mémorisation de la position, mode sombre
AJAX (Fetch API et XMLHttpRequest)	Communication asynchrone avec le serveur sans rechargement de page (progression, soumission de quiz, chargement dynamique des leçons)
PHP 8 (procédural)	Logique serveur : authentification, contrôle d'accès par rôle, traitement des formulaires, génération du HTML dynamique
PDO (PHP Data Objects)	Couche d'abstraction pour accéder à MySQL de façon sécurisée (requêtes préparées)
MySQL	Base de données relationnelle stockant utilisateurs, cours, leçons, évaluations, résultats, certificats
XAMPP	Environnement local fournissant Apache (serveur web), MySQL/MariaDB et PHP pour exécuter le projet sur la machine
Bootstrap Icons (CDN)	Bibliothèque d'icônes utilisée dans toute l'interface
Google Fonts (Inter)	Police typographique principale de l'interface

3.1 Hébergement local avec XAMPP

Le projet est conçu pour être exécuté localement via XAMPP : les fichiers PHP sont placés dans le dossier htdocs (sous le chemin /MboaLearn, comme l'indiquent les redirections `header('Location: /MboaLearn/...')`), Apache sert les pages sur `http://localhost/MboaLearn`, et MySQL/phpMyAdmin (fourni par XAMPP) héberge la base de données `l2_lms` définie dans `config/database.php` (`DB_HOST = localhost`, `DB_USER = root`, `DB_PASS` vide par défaut). Le script `index.sql` doit être importé dans phpMyAdmin pour créer les tables avant utilisation.

4. Fonctions et mécanismes principaux utilisés

4.1 Fonctions PHP / d'authentification (includes/auth.php)

Fonction	Description
<code>is_logged_in()</code>	Retourne vrai si une session utilisateur existe (<code>isset(\$_SESSION['user_id'])</code>)
<code>has_role(\$role)</code>	Vérifie que le rôle de l'utilisateur connecté correspond à <code>\$role</code>
<code>require_login()</code>	Redirige vers <code>login.php</code> si l'utilisateur n'est pas connecté
<code>require_role(\$role)</code>	Redirige vers <code>dashboard.php</code> si l'utilisateur n'a pas le rôle requis
<code>get_user_info()</code>	Récupère (id, nom, email, rôle) de l'utilisateur via une requête PDO préparée
<code>clean_input(\$data)</code>	Nettoie une entrée utilisateur (<code>trim</code> , <code>strip_tags</code> , <code>htmlspecialchars</code>) pour limiter les injections XSS

4.2 Fonctionnalités côté étudiant

- `login.php` / `register.php` : authentification avec `password_hash()`/`password_verify()`, validation des champs, gestion des messages d'erreur.
- `dashboard.php` : tableau de bord avec statistiques (nombre de cours, leçons terminées, certificats), calculées via des requêtes SQL avec `LEFT JOIN`, `COUNT()` et agrégations conditionnelles (`CASE WHEN`).
- `cours.php` : liste des cours avec recherche dynamique (paramètre GET 'search', requête SQL avec `LIKE` sur plusieurs colonnes).
- `lecon.php` : affichage d'une leçon (vidéo ou PDF), avec mise à jour automatique du statut de progression à 'en_cours' via une requête `INSERT ... ON DUPLICATE KEY UPDATE`.
- `evaluation.php` : affichage d'un QCM, vérification si l'étudiant l'a déjà passé, soumission via AJAX (`fetch`).
- `resultats.php` : affichage du score obtenu, des bonnes/mauvaises réponses, et du certificat éventuel.
- `certificats.php` / `certificate.php` : liste et affichage détaillé des certificats obtenus.

4.3 Fonctionnalités AJAX (dossier ajax/)

Endpoint	Méthode	Rôle
----------	---------	------

ajax/update_progress.php	POST	Met à jour le statut d'une leçon (terminé / en cours) pour l'étudiant connecté, via une requête INSERT ... ON DUPLICATE KEY UPDATE ; retourne du JSON
ajax/submit_quiz.php	POST	Enregistre les réponses au QCM, calcule la note, met à jour la progression, génère un certificat si la note $\geq 50\%$, retourne un JSON détaillé (note, bonnes réponses, certificat éventuel)
ajax/get_lessons.php	GET	Renvoie en JSON la liste des leçons d'un cours avec leur statut de progression pour l'étudiant connecté

Côté client, lecon.php utilise fetch() pour appeler update_progress.php, et evaluation.php utilise fetch() avec un objet FormData pour soumettre le QCM à submit_quiz.php. Le fichier assets/js/app.js fournit également une fonction générique ajaxRequest() basée sur XMLHttpRequest, illustrant les deux approches AJAX (Fetch API moderne et XMLHttpRequest classique).

4.4 Fonctions JavaScript principales (assets/js/app.js)

Fonction	Description
initAlertes()	Fait disparaître automatiquement les messages d'alerte après 5 secondes (fondu en opacité puis suppression du DOM)
initFormulaires()	Valide côté client les champs obligatoires de tous les formulaires avant soumission
initVideoPlayer()	Sauvegarde et restaure la position de lecture d'une vidéo via localStorage
showNotification(message, type)	Affiche une notification temporaire stylisée (succès, erreur, avertissement, info)
ajaxRequest(url, method, data)	Fonction générique de requête AJAX basée sur XMLHttpRequest, retournant une Promise
formatDate(dateStr)	Formate une date au format français long (jour, mois, année, heure)
copyToClipboard(text)	Copie un texte dans le presse-papiers via l'API navigator.clipboard
toggleDarkMode()	Active/désactive un mode sombre, mémorisé via localStorage

4.5 Espaces Enseignant et Promoteur

Le dossier enseignant/ contient les pages permettant à un enseignant de gérer ses cours (ajouter_cours.php, mes_cours.php), ses leçons (ajouter_lecon.php, mes_lecons.php) et ses évaluations/QCM (ajouter_evaluation.php, mes_evaluations.php). Le dossier promoteur/ propose un tableau de bord global avec des statistiques (nombre d'étudiants, d'enseignants, de modules, de cours, de certificats), la gestion des modules (promoteur/modules.php) et des

statistiques détaillées (promoteur/statistiques.php). L'accès à ces espaces est protégé par `require_role('enseignant')` ou `require_role('promoteur')`.

4.6 Composants réutilisables (includes/)

- `includes/header.php` : en-tête commun (navigation, menu adapté au rôle, lien de recherche, informations utilisateur, bouton de déconnexion), inclus via `include 'includes/header.php'`.
- `includes/footer.php` : pied de page commun affichant le copyright, le nom de l'UE et les informations de l'étudiant(e) (Yvana Emilia Lalane Larcier — matricule 24G2439).

5. Flux fonctionnel global (workflow étudiant)

1. Inscription (`register.php`) ou connexion (`login.php`) avec hachage/vérification du mot de passe.
2. Redirection vers `dashboard.php`, qui aiguille l'utilisateur selon son rôle.
3. L'étudiant consulte le catalogue de cours (`cours.php`), avec recherche possible.
4. Il ouvre une leçon (`lecon.php`) : la progression passe à 'en_cours', puis l'étudiant peut la marquer 'terminé' via AJAX (`ajax/update_progress.php`).
5. Si la leçon possède une évaluation, l'étudiant accède à `evaluation.php` et soumet ses réponses via AJAX (`ajax/submit_quiz.php`).
6. Le score est calculé côté serveur ; si la note $\geq 50\%$, un certificat est généré automatiquement dans la table `certificats`.
7. L'étudiant consulte ses résultats (`resultats.php`) et ses certificats (`certificats.php`, `certificate.php`).

6. Ce que ce TP m'a permis d'apprendre

6.1 Sur le plan PHP / back-end

- Mettre en place une connexion sécurisée à MySQL avec PDO (mode `FETCH_ASSOC`, gestion des exceptions, requêtes préparées).
- Structurer un projet PHP procédural en séparant configuration, authentification, vues et endpoints AJAX.
- Gérer des sessions PHP (`$_SESSION`) pour l'authentification et le contrôle d'accès basé sur les rôles.
- Sécuriser les mots de passe avec `password_hash()` et `password_verify()`, et nettoyer les entrées utilisateurs (`clean_input`).
- Écrire des requêtes SQL plus avancées : jointures multiples (`LEFT JOIN`, `JOIN`), agrégations (`COUNT`, `AVG`), expressions conditionnelles (`CASE WHEN`), et la clause `INSERT ... ON DUPLICATE KEY UPDATE` pour gérer l'insertion/mise à jour en une seule requête.
- Concevoir un schéma relationnel normalisé avec clés étrangères reliant utilisateurs, cours, leçons, évaluations, résultats et certificats.

6.2 Sur le plan front-end (HTML/CSS/JS)

- Construire des interfaces dynamiques générées côté serveur (PHP injecté dans le HTML) tout en gardant une structure HTML claire.
- Utiliser CSS pour créer des composants réutilisables (cartes, grilles responsives, barres de progression, badges).
- Manipuler le DOM en JavaScript pur (sans framework) pour la validation de formulaires et les notifications.
- Utiliser localStorage pour conserver des préférences côté client (position vidéo, mode sombre) entre les sessions de navigation.

6.3 Sur le plan AJAX

- Comprendre la différence entre une requête classique (rechargement de page) et une requête asynchrone (AJAX).
- Implémenter AJAX avec deux approches : la Fetch API moderne (fetch().then()) et XMLHttpRequest classique.
- Faire communiquer le front-end et le back-end via des endpoints PHP dédiés renvoyant du JSON (header('Content-Type: application/json')).
- Mettre à jour l'interface en temps réel (progression, résultats de quiz) sans recharger la page.

6.4 Sur le plan environnement de développement

- Configurer et utiliser XAMPP pour héberger localement un projet PHP/MySQL (Apache + phpMyAdmin).
- Importer un schéma SQL (index.sql) et appliquer une migration (migration_certificats.sql) dans phpMyAdmin.
- Tester une application multi-rôles avec des comptes de démonstration (administrateur, enseignant, étudiant).

7. Conclusion

MboaLearn constitue une application web complète et fonctionnelle qui couvre l'ensemble des notions clés de l'UE INF222 : structuration HTML/CSS, dynamisme PHP côté serveur, gestion d'une base de données relationnelle via PDO, sécurité de base (sessions, hachage de mots de passe, requêtes préparées), et interactivité côté client via JavaScript et AJAX. Ce TP a permis de consolider une vision globale du développement web full-stack, depuis la conception de la base de données jusqu'à l'expérience utilisateur finale, dans un environnement d'hébergement local réaliste (XAMPP).

Document préparé par : Yvana Emilia Lalane Larcier (Matricule : 24G2439)

UE : INF222 — Programmation Web | **Examineur :** Dr Messi Nguele